

Security Audit

FX1 (Sports AI)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Status Definitions	13
Audit Findings	14
Centralisation	32
Conclusion	33
Our Methodology	34
Disclaimers	36
About Hashlock	37

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The FX1 team partnered with Hashlock to conduct a security audit of their FX1Staking.sol smart contract. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

FX1 is a next-generation sports technology company that revolutionizes combat sports analytics by integrating AI, blockchain, and advanced data. It enhances fan engagement, sports betting, and broadcasting by providing real-time insights, AI-driven predictions, and immersive experiences. FX1 also pioneers a decentralized sports data ecosystem, ensuring accuracy, transparency, and monetization through blockchain-backed validation. With a focus on MMA, boxing, jiu-jitsu, and other combat sports, FX1 transforms how leagues and audiences interact with sports data.

Project Name: FX1

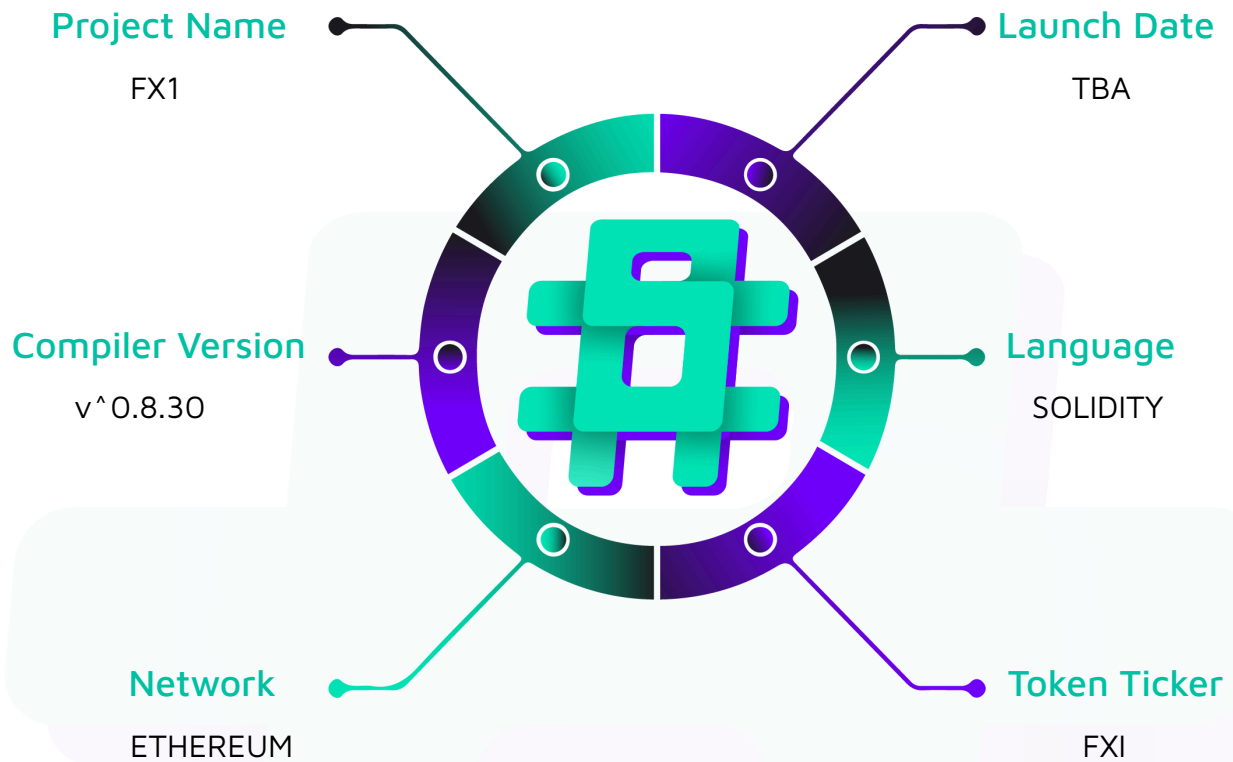
Project Type: dApp

Compiler Version: ^0.8.30

Website: <https://fx1.io/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the FX1 project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	FX1 Smart Contracts
Platform	Ethereum / Solidity
Audit Date	June, 2025
Contract 1	FX1Staking.sol
Contract 1 MD5 Hash	843a15c2ce491580fe896d02f0e8bd11
Audited GitHub Commit Hash	035b15012a9bbeebae149bb971a260be611405a3
Fix Review GitHub Commit Hash	fa1303eabc656e8a0532b265c3ae470b80636491

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

5 Medium severity vulnerabilities

4 Low severity vulnerabilities

3 Gas Optimisations

1 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
FXIStaking.sol Core Staking Operations 1. Tier-based staking - Users stake tokens into 14-day, 35-day, or 70-day duration tiers - Tier selection determines reward rates and lock-up periods 2. Dual Reward System - ETH rewards are distributed proportionally based on tier allocations - Reward tokens are calculated via fixed APYs per tier 3. Restaking Mechanism - Restake pending unstaked tokens back into the current tier - Restake the entire principal into the same/different tier after the lock-up period 4. Unstaking Process - Initiate unstake to move tokens to the pending state - Withdraw unstaked tokens after the tier duration expires Reward Management 1. ETH Reward Distribution - Public ETH deposits distributed to tiers (5%/15%/80% allocation) - Unallocated ETH stored for future distribution	Contract achieves this functionality.

- Rewards are tracked via accumulated rewards per share

2. APY-based Token Rewards

- Fixed APYs: 2% (14-day), 10% (35-day), 15% (70-day)
- Continuously accrues based on the staked amount and time

3. Reward Claiming

- Separate functions for claiming ETH and token rewards
- ETH claims paid from contract balance
- Token claims transferred from the designated token vault

Administrative Functions

1. Owner Controls
 - Distribute unallocated ETH rewards
 - Emergency withdraw all contract ETH (creates debt)
 - Update the reward token address
 - Change the token vault address

View Functions

1. State Getters
 - Query claimable ETH/token rewards
 - Check unstaked token availability
 - View tier allocations and global stats
 - Get user-specific stake details

Code Quality

This audit scope involves the smart contracts of the FX1 project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices to help avoid unnecessary complexity that increases the likelihood of exploitation; however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the FX1 project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] FXStaking.sol - Incorrect Default Tier for Non-Stakers

Description

The contract initializes new users with an active staking tier (TIER_14_DAYS) instead of the NONE tier. This violates the intended design where users without stakes should have no active tier.

Vulnerability Details

```
enum StakingTier {  
    TIER_14_DAYS, // Index 0 (default)  
    TIER_35_DAYS, // Index 1  
    TIER_70_DAYS, // Index 2  
    NONE          // Index 3  
}
```

Solidity enums default to their first value (index 0) for uninitialized variables. The enum definition places TIER_14_DAYS at index 0 and NONE at index 3. New users without prior interactions have currentTier implicitly set to TIER_14_DAYS (0) rather than NONE (3).

Impact

Non-stakers are misclassified as active stakers, breaking core contract invariants.

Recommendation

Reorder StakingTier Enum to make NONE the default value(index 0):

```
enum StakingTier {  
    NONE, // Index 0 (default)  
    TIER_14_DAYS, // Index 1  
    TIER_35_DAYS, // Index 2  
    TIER_70_DAYS // Index 3  
}
```

```
}
```

Also, remove the following line in the constructor:

```
constructor(  
    address _stakingToken,  
    address _rewardToken,  
    address _tokenVaultAddr  
) Ownable(msg.sender) {  
    if (_stakingToken == address(0))  
        revert("Staking token cannot be zero address");  
    if (_rewardToken == address(0))  
        revert("Reward token cannot be zero address");  
    if (_tokenVaultAddr == address(0))  
        revert("Token vault cannot be zero address");  
  
    stakingToken = IERC20(_stakingToken);  
    rewardToken = IERC20(_rewardToken);  
    tokenVault = _tokenVaultAddr;  
    -- userInfo[address(0)].currentTier = StakingTier.NONE;  
}
```

Status

Resolved

[M-02] FXIStaking.sol - Potential Denial of Service via rewardToken or tokenVault Changes

Description

The owner of the contract can call `setRewardToken` or `setTokenVault` to change the reward token or vault address at any time. If changed to an incompatible token or an empty vault, users may be unable to claim FXI rewards until corrected.

Vulnerability Details

The function `claimFxiRewards` will attempt to transfer token rewards to users from `tokenVault` via `SafeERC20 safeTransferFrom` function. This operation will fail if the owner of the contract calls `setRewardToken` or `setTokenVault` to change the reward token to an incompatible token or vault address to an empty vault, causing denial of service to stakers.

Impact

This could temporarily or permanently block reward claims, especially if the new `tokenVault` lacks funds or approvals, effectively denying service to stakers.

Recommendation

Verify the new `tokenVault` has sufficient balance and approval in the constructor and the `setTokenVault` function. Additionally, add a timelock or delay mechanism for these changes, allowing users to react (claim rewards before the switch).

Note

FX1's team informed Hashlock that this finding relates to a process that will be handled offchain initially by notifying their community with a deadline, and then executed onchain at the team's decided time and date.

Status

Acknowledged

[M-03] FXIStaking.sol - Reward Token Change Might Break Accounting

Description

The `setRewardToken` function allows the owner to change the ERC20 reward token without migrating existing `claimableFxiRewards`. User rewards accrued in the old token will be paid in the new token, which may have a different value or decimals.

Vulnerability Details

```
function setRewardToken(address _newRewardToken) external onlyOwner {  
    rewardToken = IERC20(_newRewardToken); // No migration of existing rewards  
}
```

The owner can change the reward token at any point in time by calling `setRewardToken` function. However, if the new reward token has a different decimal, it will break the accounting.

Impact

Users receive incorrect reward amounts (e.g., 100 old tokens vs. 100 new tokens with different decimals), leading to loss of funds.

Recommendation

Disallow changing the reward token after deployment, or force users to claim existing rewards in the old token before switching.

Note

FX1's team informed Hashlock that this finding relates to ensuring that the new reward token will match the decimal precision of the original token. The team confirmed that this will be the case.

Status

Acknowledged

[M-04] FXStaking.sol - Emergency Withdrawal Fairness and Transparency

Description

The `emergencyWithdrawUnclaimedRewards` function allows the owner to withdraw all ETH, increasing `ethDebt`. Future ETH deposits repay this debt before distribution, but this process lacks user visibility and could be abused if the owner frequently withdraws funds.

Vulnerability Details

The function `emergencyWithdrawUnclaimedRewards` is designed to allow the contract owner to withdraw all ETH from the contract's balance during an emergency, such as a security breach or urgent liquidity need. While this feature is practical, it introduces vulnerabilities related to fairness and transparency that could impact users.

Impact

Users might not realize their deposited ETH is being used to repay `ethDebt` rather than rewarding stakers, potentially eroding trust. Also, there's no limit on how often the owner can call this function, and there is no cap on the `ethDebt` amount, meaning the owner could withdraw an arbitrarily large sum, limited only by the contract's ETH balance.

Recommendation

Implement a cooldown period or cap on `ethDebt` to prevent excessive withdrawals. Also, add an event such as `EmergencyWithdrawal(uint256 amount, uint256 ethDebt)` that triggers whenever the function is called for transparency.

Note

FX1's team informed Hashlock that this finding relates to the implementation of emergency withdrawal functionality. The team clarified that applying any restrictions would contradict the purpose of an emergency withdrawal. The event was added solely to ensure transparency.

Status

Acknowledged

[M-05] FXIStaking.sol - Incompatibility with fee-on-transfer FXI staking token

Description

Fee-on-transfer tokens deduct a fee during transfers, meaning the actual amount received by the contract is less than the amount specified in the transaction. The current implementation assumes the full amount is deposited into the contract, leading to accounting inaccuracies. This mismatch between recorded and actual balances creates vulnerabilities in fund tracking and fee calculations.

Vulnerability Details

In `FXIStaking::stake()` function, `user.stakedAmount` variable which tracks the stake balance assumes exact transferred amounts, but the fee-on-transfer staking token `FXI` reduces received amounts, causing accounting errors.

In `FXIStaking.sol::stake` function:

```
function stake(uint256 _amount, uint8 _days) external nonReentrant {
    UserInfo storage user = userInfo[msg.sender];
    if (_amount == 0) {
        revert("Cannot stake 0 tokens");
    }
    StakingTier newTier = _getStakingTierFromDays(_days);

    _updateFxiRewards(msg.sender); // Calculate and bank pending FXI rewards before
state changes

    // 1. Crystallize ETH rewards and update totals if user is already staked
    if (user.stakedAmount > 0) {
        _crystallizeEthRewards(msg.sender);
        totalStakedAmountTier[user.currentTier] -= user.stakedAmount;
        totalStakedAmount -= user.stakedAmount;
    }
}
```

```

// 3. Update user's total staked amount (state change before external call)
user.stakedAmount += _amount;

// 4. Update user's tier and lock time for the *entire* stake
user.currentTier = newTier;
user.stakeUnlockTime =
    block.timestamp +
    _getTierDurationInSeconds(newTier);

// 5. Add the user's *total current* staked amount to the new tier's accounting
totalStakedAmountTier[newTier] += user.stakedAmount;
totalStakedAmount += user.stakedAmount;

// 6. Update ETH reward debt for the new tier and total new stake
user.rewardDebt =
    (user.stakedAmount * accRewardsPerShareTier[newTier]) /
    REWARD_PRECISION;

// 7. Reset FXI reward timer for the new/updated stake
user.lastFxiRewardUpdateTime = block.timestamp;

// Perform the token transfer (interaction) last
stakingToken.safeTransferFrom(msg.sender, address(this), _amount);

emit Stake(msg.sender, _amount, newTier);
}

```

If `_amount = 100` and FXI staking token has a 5% fee, the contract will only receive 95 FXI tokens, but `user.stakedAmount` is increased by 100, leading to an overstatement.

Impact

FXI staking token is a fee-on-transfer token; the balance after the transfer might be smaller than expected, leading to accounting issues.

Recommendation

Exclude `FXIStaking` contract from fees via `FXI::excludeWalletsFromFees` function.

Note

FX1's team informed Hashlock that this finding relates to fee exclusions. The team confirmed that **FX1staking** will be added to the fee exclusion list upon deployment.

Status

Acknowledged



Low

[L-01] FXStaking.sol - Unrestricted renounceOwnership() Call

Description

The contract inherits from OpenZeppelin's `Ownable` contract, which includes the `renounceOwnership()` function. This function allows the current owner to irreversibly relinquish ownership, setting the owner address to `address(0)`.

Impact

In the current implementation, `renounceOwnership()` is not overridden or restricted, meaning any accidental or intentional call by the owner will permanently remove owner control over the contract.

Recommendation

If ownership should never be renounced (common for fixed-governance or permanently-administered tokens), override and disable it by adding the following function:

```
function renounceOwnership() public override onlyOwner {  
    revert("Renouncing ownership is disabled.");  
}
```

Status

Resolved

[L-02] FXIStaking.sol - Use Ownable2Step contract rather than Ownable contract

Description

Ownable contract from OpenZeppelin is used in the FXIStaking.sol contract. The primary concern with the Ownable contract's transferOwnership function is its immediacy and lack of verification. Once called, ownership is instantly transferred to the specified address without any confirmation from the recipient.

Impact

If the new owner's address is entered incorrectly, the contract's control could be lost or handed over to an unintended entity, leading to potential security risks or loss of functionality.

Recommendation

To mitigate this risk, it's advisable to use OpenZeppelin's Ownable2Step contract, this contract enhances the ownership transfer process by implementing a two-step mechanism.

This two-step process ensures that ownership is only transferred if the new owner explicitly accepts it, thereby preventing accidental transfers to incorrect addresses.

Status

Resolved

[L-03] FXIStaking.sol - Important functions lack event emissions

Description

In Solidity, events provide a critical logging mechanism that external observers (like dApps, users, or auditors) rely on to track what's happening on-chain. Important functions, such as those that modify contract state, transfer funds, or change ownership, should emit events to provide transparency.

The following critical functions were lacking in event emissions:

```
function refreshPool()  
  
function emergencyWithdrawUnclaimedRewards()
```

Impact

Critical actions (e.g., withdrawals, admin changes) occur silently, making monitoring via block explorers or off-chain tools impossible.

Recommendation

Emit appropriate events in all important state-changing functions.

Status

Resolved

[L-04] FXIStaking.sol - No Check for Zero Amount Distribution in claimEthRewards

Description

The `claimEthRewards` function does not check if `amountToDistribute > 0` before attempting to send ETH. If the contract's balance is zero, it sends `amountToDistribute = 0`, which is unnecessary and might cause confusion for users.

Impact

While not a direct security issue, this leads to inefficient gas usage and could confuse users expecting a revert when no ETH is available.

Recommendation

Add the check `if (amountToDistribute == 0) revert("No ETH available to claim");` before the transfer to avoid pointless operations.

Status

Resolved



Gas

[G-01] **FXIStaking.sol** - Code Optimisation in emergencyWithdrawUnclaimedRewards function

Description

The code updates `ethDebt` before the ETH transfer (`ethDebt += amount`). If the transfer fails, it attempts to revert the state by subtracting the amount (`ethDebt -= amount`) before reverting the transaction.

In solidity, the revert undoes all state changes, making the manual subtraction (`ethDebt -= amount`) redundant. This wastes gas (expensive `SSTORE` operation) for a state change that will be discarded.

Recommendation

Refactor the code as follows:

```
function emergencyWithdrawUnclaimedRewards()
    external
    onlyOwner
    nonReentrant
{
    uint256 amount = address(this).balance;
    if (amount == 0) {
        revert("No ETH in contract to withdraw");
    }
    -- ethDebt += amount;
    (bool sent, ) = msg.sender.call{value: amount}("");
    if (!sent) {
    -- ethDebt -= amount;
        revert("ETH transfer failed during emergency withdrawal");
    }
    ++ ethDebt += amount;
}
```

Status

Resolved

[G-02] FXIStaking.sol - Code Optimisation in claimEthRewards function

Description

The code updates `user.claimableRewards` and `user.claimedAmount` before the ETH transfer. If the transfer fails, it attempts to revert the state by subtracting the `user.claimedAmount` and adding back the `user.claimableRewards` before reverting the transaction.

In solidity, the revert undoes all state changes, making the manual addition and subtraction redundant. This wastes gas (expensive `SSTORE` operation) for a state change that will be discarded.

Recommendation

Refactor the code as follows:

```
function claimEthRewards() external nonReentrant {
    UserInfo storage user = userInfo[msg.sender];

    if (user.stakedAmount == 0 && user.claimableRewards == 0) {
        revert(
            "No ETH rewards to claim or no active stake for pending calculation"
        );
    }
    if (user.currentTier == StakingTier.NONE && user.stakedAmount > 0) {
        revert("User in invalid tier with active stake");
    }

    _crystallizeEthRewards(msg.sender); // This will add pending from active stake to
    user.claimableRewards
    uint256 totalClaimable = user.claimableRewards;

    if (totalClaimable == 0) {
        revert("No ETH rewards to claim");
    }
}
```

```

    if (user.stakedAmount > 0) {
        user.rewardDebt =
            (user.stakedAmount * accRewardsPerShareTier[user.currentTier]) /
            REWARD_PRECISION;
    }

    uint256 amountToDistribute = address(this).balance >= totalClaimable
        ? totalClaimable
        : address(this).balance;
--    user.claimableRewards -= amountToDistribute; // Subtract what's being sent
--    user.claimedAmount += amountToDistribute;

    (bool sent, ) = msg.sender.call{value: amountToDistribute}("");
    if (!sent) {
        // Revert state changes made for this claim
--        user.claimedAmount -= amountToDistribute;
--        user.claimableRewards += amountToDistribute; // Add back if send failed
        // Note: pendingFromActiveStake was already added to claimableRewards,
        // and rewardDebt was updated. Reverting these precisely is complex if not
full revert.
        // A full revert is cleaner.
        revert("ETH transfer failed");
    }
++    user.claimableRewards -= amountToDistribute; // Subtract what's being sent
++    user.claimedAmount += amountToDistribute;
    emit Claim(msg.sender, amountToDistribute);
}

```

Status

Resolved

[G-03] FXIStaking.sol - Late Zero-Address Checks in claimFxiRewards

Description

The `claimFxiRewards` function checks if `rewardToken` or `tokenVault` is a zero address after updating the state, wasting gas on unnecessary state changes if the check fails.

Recommendation

Refactor the code as follows:

```
function claimFxiRewards() external nonReentrant {
--    _updateFxiRewards(msg.sender); // Calculate and bank all pending FXI rewards
++    if (address(rewardToken) == address(0)) {
++        revert("Reward token not set");
++    }
++    if (tokenVault == address(0)) {
++        revert("Token vault not set");
++    }
++    _updateFxiRewards(msg.sender); // Calculate and bank all pending FXI rewards
    UserInfo storage user = userInfo[msg.sender];
    uint256 amountToClaim = user.claimableFxiRewards;

    if (amountToClaim == 0) {
        revert("No FXI rewards to claim");
    }
--    if (address(rewardToken) == address(0)) {
--        revert("Reward token not set");
--    }
--    if (tokenVault == address(0)) {
--        revert("Token vault not set");
--    }
    user.claimableFxiRewards = 0;
    user.claimedFxiAmount += amountToClaim;

    // This contract needs approval from tokenVault to transfer 'amountToClaim' of
rewardToken
    rewardToken.safeTransferFrom(tokenVault, msg.sender, amountToClaim);

    emit ClaimFxiRewards(msg.sender, amountToClaim);
}
```

}

Status

Resolved



QA

[Q-01] FXIStaking.sol - Floating pragma

Description

The contract `FXIStaking.sol` uses a floating pragma version (^0.8.30). It's crucial to compile and deploy contracts with the same compiler version and settings used during development and testing. Fixing the pragma version prevents compilation with different versions. Using an outdated pragma version could introduce bugs that negatively affect the contract system, while newly released versions may have undiscovered security vulnerabilities.

Recommendation

Change the pragma statement in both contracts to a fixed version, such as `pragma solidity 0.8.30`. This locks the contract to a specific compiler version.

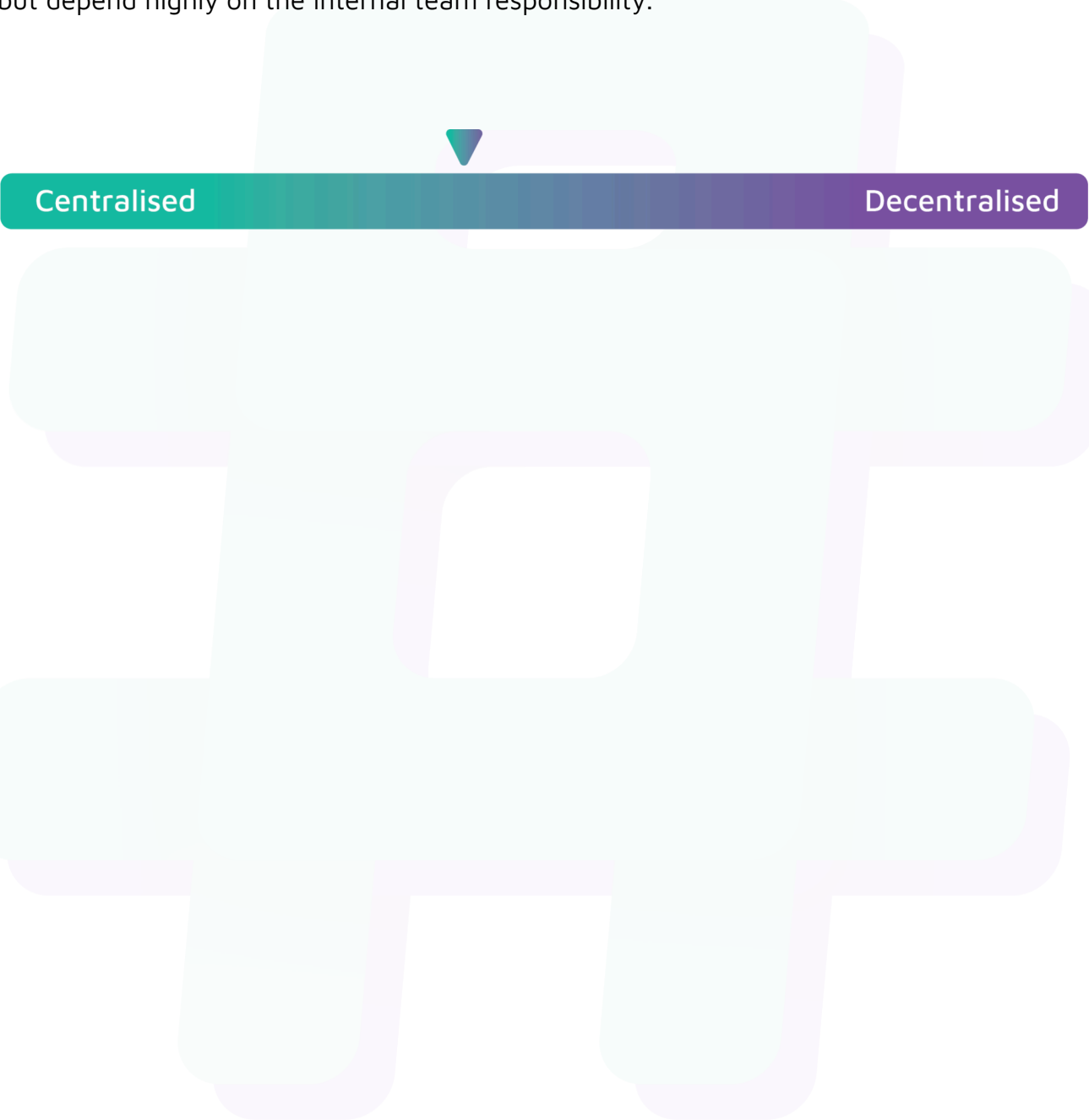
Status

Resolved

Centralisation

The FX1 project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality, but depend highly on the internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the FX1 project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.